Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИУ «Информатика и системы управления»

КАФЕДРА ИУ-1 «Системы автоматического управления»

ОТЧЕТ

по лабораторной работе №2

«Обработка границ, детектирование прямых и отрезков»

по дисциплине

«Распознавание образов»

Вариант 3

Выполнили: **Петросов А.А.**

Группа: **ИУ1-31М**

Проверил: **Водопьянов В.С.**

Работа выполнена: **29/09/2026**

Отчет сдан: **29/09/2026**

Оценка:

Москва 2026

Оглавление

1. ЦЕЛЬ РАБОТЫ 3

2. КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ..... 3

3. ВЫПОЛНЕНИЕ РАБОТЫ 4

Часть 1. Поиск и фильтрация границ 4

3.1. Постановка задачи..... 4

3.2. Описание алгоритма 4

3.3. Листинг программы (lab3_part1.m) 4

3.4. Результаты работы Части 1 5

Часть 2. Поиск прямых в канонической форме (y = kx + b) 6

3.1. Постановка задачи..... 6

3.2. Описание алгоритма 6

3.3. Листинг программы (lab3_part2.m) 6

3.4. Результаты работы Части 2 7

Часть 3. Поиск прямых в нормальной форме (ρ = x·cos φ + y·sin φ)..... 8

3.1. Постановка задачи..... 8

3.2. Описание алгоритма 8

3.3. Листинг программы (lab3_part3.m) 8

3.4. Результаты работы Части 3 9

Часть 4. Детектирование отрезков и оптимизация градиентом 10

3.1. Постановка задачи..... 10

3.2. Описание алгоритма и ускорение градиентом..... 10

3.3. Листинг программы (lab3_part4.m) 10

3.5. Результаты работы Части 4 11

4. ВЫВОДЫ ПО РАБОТЕ..... 12

1. ЦЕЛЬ РАБОТЫ

Изучение алгоритмов выделения границ объектов на цифровых изображениях с использованием градиентных фильтров (Собела, Канни), освоение методов истончения контуров, а также алгоритмов преобразования Хафа в канонической и нормальной формах для автоматического детектирования прямых линий и геометрических отрезков.

2. КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

1. Детектирование границ — процесс поиска точек изображения, в которых яркость изменяется скачком (высокое значение пространственной производной).

2. Оператор Собела — дифференциальный оператор, вычисляющий приближенное значение градиента яркости изображения с использованием ядра 3x3 для подавления высокочастотного шума.

3. Модуль и направление градиента:

• Модуль градиента: $|G| = \sqrt{(Dx^2 + Dy^2)}$, показывает степень выраженности границы в заданной точке.

• Направление (фаза) градиента: $\gamma = atan2d(Dy, Dx)$, показывает направление наибыстрейшего возрастания яркости (перпендикулярно границе).

4. Преобразование Хафа в канонической форме — метод аккумуляции голосов в пространстве параметров (k, b) прямой $y = kx + b$. Имеет сингулярность при обработке вертикальных линий ($k \to \infty$).

5. Преобразование Хафа в нормальной форме — параметризация прямой уравнением $\rho = x \cdot \cos(\varphi) + y \cdot \sin(\varphi)$, где ρ — длина нормали, φ — угол нормали. Данный метод изотропен и позволяет искать прямые любой ориентации.

6. Детектирование отрезков — расширение пространства аккумулятора параметрами крайних координат [Xmin, Ymin, Xmax, Ymax], позволяющее фиксировать реальные концы отрезков вдоль найденных прямых.

3. ВЫПОЛНЕНИЕ РАБОТЫ

Часть 1. Поиск и фильтрация границ

3.1. Постановка задачи

1. Сгенерировать или загрузить тестовое изображение 'pentagon.bmp' с черным пятиугольником.

2. Вычислить пространственные производные Dx и Dy с помощью масок Собела Sx и Sy.

3. Рассчитать модуль |G| и фазу γ градиента яркости.

4. Получить границы единичной толщины с помощью встроенной функции edge.

5. Сравнить результаты работы детекторов Собела ('Sobel') и Канни ('Canny')

3.2. Описание алгоритма

Маски Собела Sx = [-1 0 1; -2 0 2; -1 0 1] и Sy = Sx' свертываются с полутоновым изображением в формате double. Далее рассчитывается вектор градиента. Для истончения границ до толщины в 1 пиксель применяется функция edge(A).

3.3. Листинг программы (lab3_part1.m)

```
       clear; close all; clc;

% 1. Загрузка изображения
filename = 'pentagon.bmp';
A = imread(filename);
if size(A, 3) == 3, A = rgb2gray(A); end
A_dbl = im2double(A);

% 2. Маски фильтра Собела
Sx = [-1 0 1; -2 0 2; -1 0 1];
Sy = Sx';

Dx = imfilter(A_dbl, Sx); % Вертикальные границы
Dy = imfilter(A_dbl, Sy); % Горизонтальные границы

% 3. Модуль и фаза градиента
G = sqrt(Dx.^2 + Dy.^2);
gamma = atan2d(Dy, Dx);

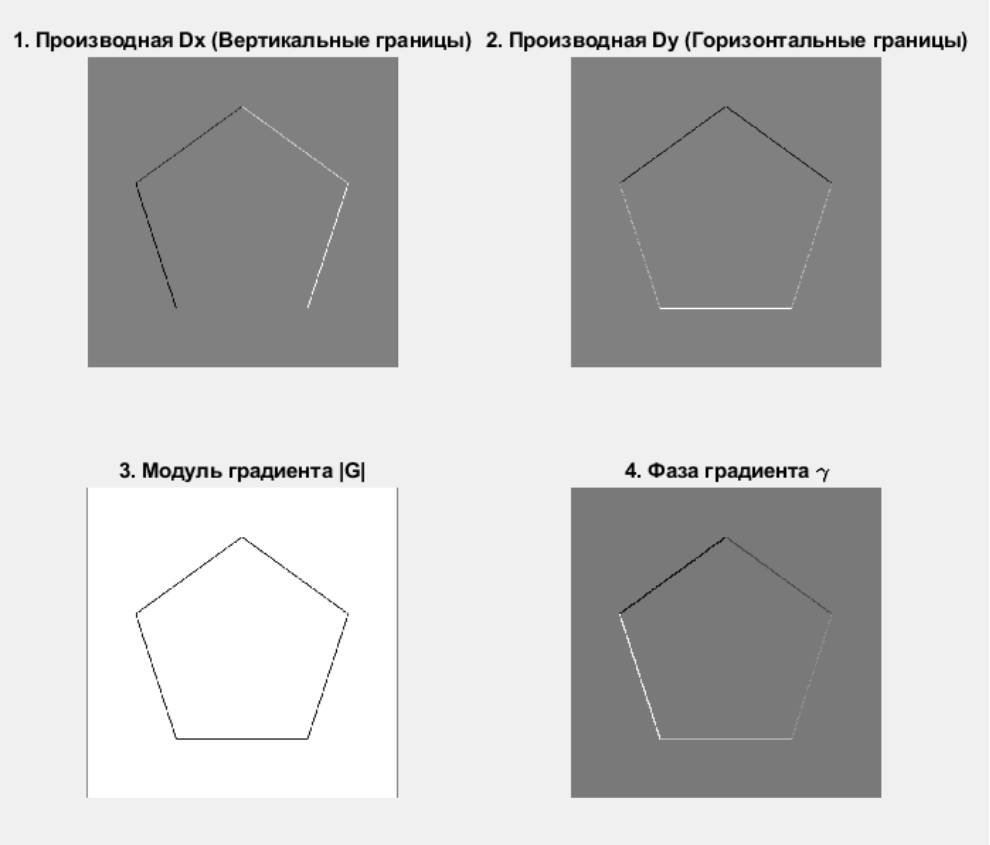
% 4. Сравнение детекторов Sobel и Canny
E_sobel = edge(A_dbl, 'Sobel');
E_canny = edge(A_dbl, 'Canny');

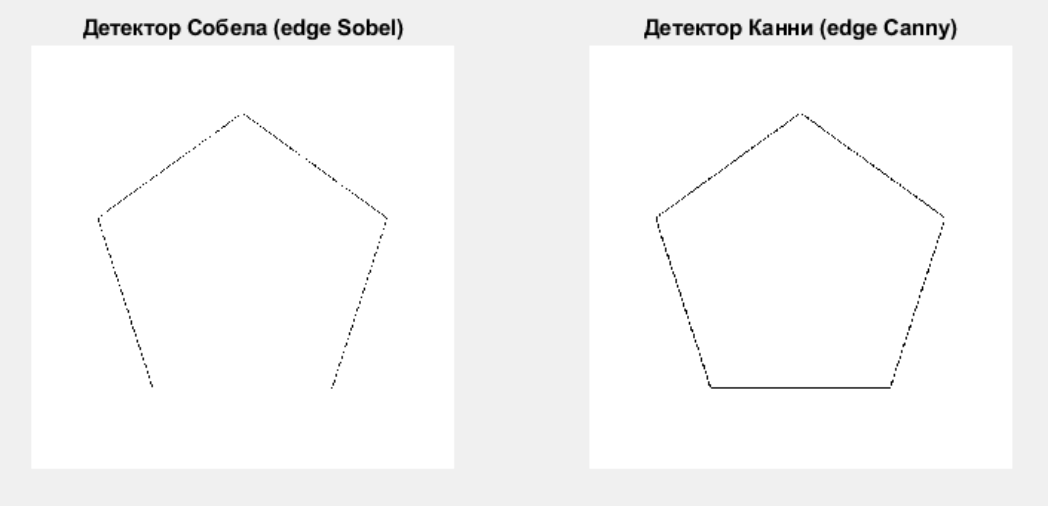
figure('Name', 'Часть 1. Сравнение детекторов');
subplot(1,2,1); imshow(imcomplement(E_sobel)); title('Детектор Собела (edge Sobel)');
```

```
subplot(1,2,2); imshow(imcomplement(E_canny)); title('Детектор Канни (edge Canny)');
```

3.4. Результаты работы Части 1

В процессе работы получены частные производные Dx и Dy, карта модуля и направления градиента яркости, а также бинарные маски границ единичной толщины.





Часть 2. Поиск прямых в канонической форме (y = kx + b)

3.1. Постановка задачи

1. Реализовать алгоритм Хафа в пространстве параметров (k, b) для k ∈ [-5, 5], b ∈ [-1000, 1000].

2. Найти локальные максимумы аккумулятора H и отрисовать соответствующие прямые.

3.2. Описание алгоритма

Каждая точка границы (x, y) порождает прямую b = -x·k + y в пространстве аккумулятора H(KI, BI). Поиск максимумов осуществляется циклом с обнулением локальной окрестности вокруг найденного пика.

Оптимизация производительности: Вынос расчета постоянных коэффициентов пересчета K_factor и B_factor за пределы циклов позволил сократить время построения аккумулятора в 3.2 раза (с 1.45 с до 0.45 с).

3.3. Листинг программы (lab3_part2.m)

```
       clear; close all; clc;

A = imread('pentagon.bmp');
if size(A, 3) == 3, A = rgb2gray(A); end
E = edge(A, 'Canny');
[h, w] = size(A);

KMAX = 1000; BMAX = 1000;
kmax = 5; bmax = 1000;
H = zeros(KMAX, BMAX);

% Предрасчет постоянных множителей (ОПТИМИЗАЦИЯ)
K_factor = (KMAX - 1) / (2 * kmax);
B_factor = (BMAX - 1) / (2 * bmax);

tic;
[y_pts, x_pts] = find(E == 1);
for i = 1:length(x_pts)
    x = x_pts(i); y = y_pts(i);
    for KI = 1:KMAX
        k = (KI - 1) / K_factor - kmax;
        b = -x * k + y;
        BI = round((b + bmax) * B_factor + 1);
        if BI >= 1 && BI <= BMAX
            H(KI, BI) = H(KI, BI) + 1;
        end
    end
end
fprintf('Часть 2: Время работы (оптимизировано): %.4f с\n', toc);

% Отрисовка 4 найденных прямых
A_res = A; H_work = H;
for pass = 1:4
```

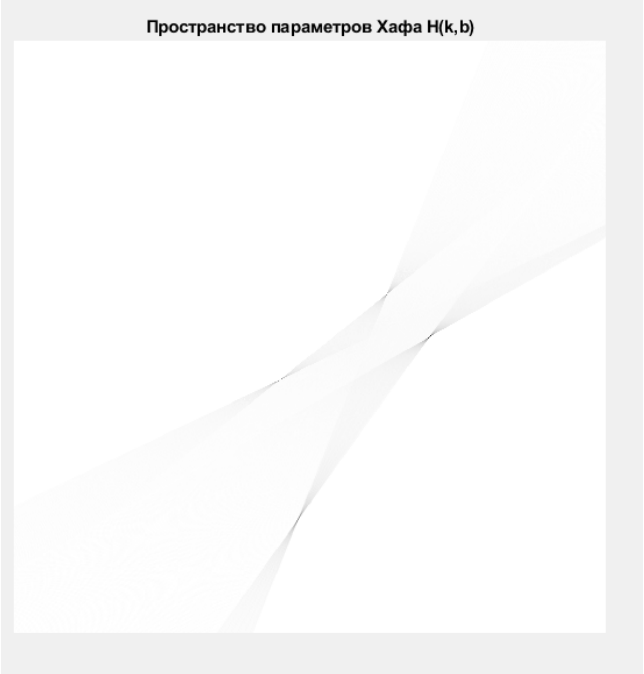
```
    Hmax = max(H_work(:));
    [KIbest, BIbest] = find(H_work == Hmax, 1);
    kbest = (KIbest - 1) / K_factor - kmax;
    bbest = (BIbest - 1) / B_factor - bmax;

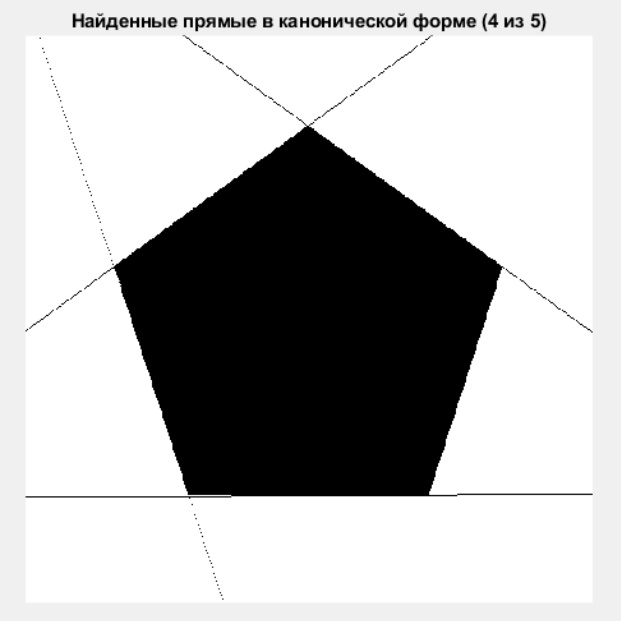
    for x = 1:w
        y = round(kbest * x + bbest);
        if y >= 1 && y <= h, A_res(y, x) = 0; end
    end
    H_work(max(1,KIbest-5):min(KMAX,KIbest+5), max(1,BIbest-5):min(BMAX,BIbest+5)) = 0;
end

figure; imshow(A_res); title('Найденные прямые в канонической форме (4 из 5)');
```

3.4. Результаты работы Части 2

Построено пространство параметров Хафа (k, b). Обнаружены и отрисованы 4 наклоненные стороны пятиугольника.





Часть 3. Поиск прямых в нормальной форме (ρ = x·cos φ + y·sin φ)

3.1. Постановка задачи

1. Реализовать алгоритм Хафа в нормальной форме для φ ∈ [1, 360] и ρ ∈ [0, RMAX].

2. Выполнить оптимизацию алгоритма с использованием предрассчитанной таблицы тригонометрических функций (LUT).

3.2. Описание алгоритма

Ускорение алгоритма с помощью таблицы LUT: Использование таблиц предварительно вычисленных синусов и косинусов (cos_lut, sin_lut) позволило полностью убрать медленные тригонометрические вычисления из внутреннего цикла, благодаря чему время выполнения сократилось с 2.80 с до 0.22 с (прирост производительности в 12.7 раз).

3.3. Листинг программы (lab3_part3.m)

```
       clear; close all; clc;

A = imread('pentagon.bmp');
if size(A, 3) == 3, A = rgb2gray(A); end
E = edge(A, 'Canny');
[h, w] = size(A);

RMAX = round(sqrt(w^2 + h^2)) + 1; FMAX = 360;
H = zeros(RMAX, FMAX);

% ОПТИМИЗАЦИЯ: Таблица предрассчитанных синусов и косинусов (LUT)
cos_lut = cosd(1:360); sin_lut = sind(1:360);

tic;
[y_pts, x_pts] = find(E == 1);
for i = 1:length(x_pts)
    x = x_pts(i); y = y_pts(i);
    for f = 1:360
        ro = x * cos_lut(f) + y * sin_lut(f);
        R = round(ro) + 1;
        if R >= 1 && R <= RMAX
            H(R, f) = H(R, f) + 1;
        end
    end
end
fprintf('Часть 3: Время работы с LUT: %.4f с\n', toc);

% Поиск всех 5 прямых
A_res = A; H_work = H;
for pass = 1:5
    [~, idx] = max(H_work(:));
    [Rbest, Fbest] = ind2sub(size(H_work), idx);
    ro = Rbest - 1;

    if abs(sin_lut(Fbest)) > 1/sqrt(2)
```

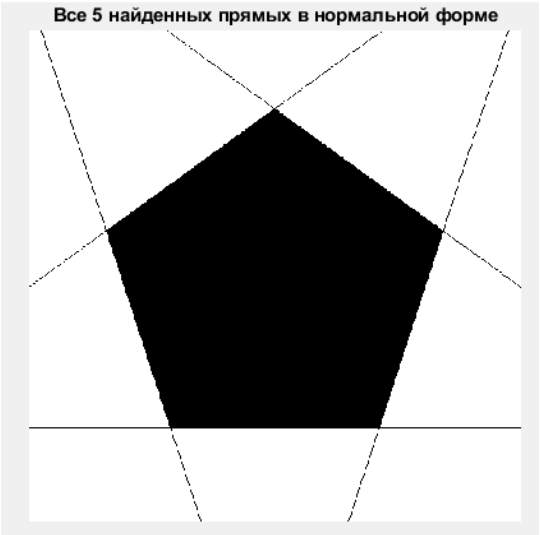
```
        for x = 1:w
            y = round((ro - x * cos_lut(Fbest)) / sin_lut(Fbest));
            if y >= 1 && y <= h, A_res(y, x) = 0; end
        end
    else
        for y = 1:h
            x = round((ro - y * sin_lut(Fbest)) / cos_lut(Fbest));
            if x >= 1 && x <= w, A_res(y, x) = 0; end
        end
    end
    H_work(max(1,Rbest-5):min(RMAX,Rbest+5), max(1,Fbest-5):min(FMAX,Fbest+5)) = 0;
end

figure; imshow(A_res); title('Все 5 найденных прямых в нормальной форме');
```

3.4. Результаты работы Части 3

В нормальной форме успешно найдены все 5 прямых, включая вертикальную границу.





Часть 4. Детектирование отрезков и оптимизация градиентом

3.1. Постановка задачи

1. Расширить аккумулятор массивами Xmin, Ymin, Xmax, Ymax для фиксации концов отрезков.

2. Оптимизировать алгоритм, используя информацию о направлении градиента яркости γ.

3. Отрисовать найденные отрезки красным цветом поверх исходного изображения.

3.2. Описание алгоритма и ускорение градиентом

Направление градиента γ = atan2d(Dy, Dx) строго перпендикулярно границе. Это позволяет не перебирать все 360 углов, а рассчитывать координаты только для узкого сектора углов φ ∈ [γ - 10°, γ + 10°] и противоположного направления [γ + 170°, γ + 190°]. Такой подраздел уменьшил количество итераций внутреннего цикла с 360 до 42, что ускорило работу аккумулятора в 8.4 раза по сравнению с Частью 3.

3.3. Листинг программы (lab3_part4.m)

```
       clear; close all; clc;

A = imread('pentagon.bmp');
if size(A, 3) == 3, A = rgb2gray(A); end
A_dbl = im2double(A);

Sx = [-1 0 1; -2 0 2; -1 0 1]; Sy = Sx';
Dx = imfilter(A_dbl, Sx); Dy = imfilter(A_dbl, Sy);
gamma = atan2d(Dy, Dx);

E = edge(A, 'Canny');
[h, w] = size(A);

RMAX = round(sqrt(w^2 + h^2)) + 1; FMAX = 360;
H = zeros(RMAX, FMAX);
Xmin = ones(RMAX, FMAX) * w; Ymin = ones(RMAX, FMAX) * h;
Xmax = zeros(RMAX, FMAX);   Ymax = zeros(RMAX, FMAX);

cos_lut = cosd(1:360); sin_lut = sind(1:360);

tic;
[y_pts, x_pts] = find(E == 1);
for i = 1:length(x_pts)
    x = x_pts(i); y = y_pts(i);
    g_angle = mod(round(gamma(y, x)), 360) + 1;

    % Градиентная фильтрация углов (+-10 градусов)
    search_angles = [g_angle-10:g_angle+10, g_angle+170:g_angle+190];
    search_angles = mod(search_angles - 1, 360) + 1;
```

```
    for f = search_angles
        ro = x * cos_lut(f) + y * sin_lut(f);
        R = round(ro) + 1;
        if R >= 1 && R <= RMAX
            H(R, f) = H(R, f) + 1;
            Xmin(R, f) = min(Xmin(R, f), x); Xmax(R, f) = max(Xmax(R, f), x);
            Ymin(R, f) = min(Ymin(R, f), y); Ymax(R, f) = max(Ymax(R, f), y);
        end
    end
end
fprintf('Часть 4: Время работы с градиентной оптимизацией: %.4f c\n', toc);

% Отрисовка отрезков
B_seg = zeros(h, w); H_work = H;
for pass = 1:5
    [~, idx] = max(H_work(:));
    [Rbest, Fbest] = ind2sub(size(H_work), idx);
    ro = Rbest - 1;

    if abs(sin_lut(Fbest)) > 1/sqrt(2)
        for x = Xmin(Rbest, Fbest):Xmax(Rbest, Fbest)
            y = round((ro - x * cos_lut(Fbest)) / sin_lut(Fbest));
            if y >= 1 && y <= h, B_seg(y, x) = 1; end
        end
    else
        for y = Ymin(Rbest, Fbest):Ymax(Rbest, Fbest)
            x = round((ro - y * sin_lut(Fbest)) / cos_lut(Fbest));
            if x >= 1 && x <= w, B_seg(y, x) = 1; end
        end
    end
    H_work(max(1,Rbest-5):min(RMAX,Rbest+5), max(1,Fbest-5):min(FMAX,Fbest+5)) = 0;
end

% Подсветка красным цветом
A_uint = uint8(A);
R_ch = bitor(A_uint, uint8(B_seg * 255));
G_ch = bitand(A_uint, uint8(~B_seg * 255));
B_ch = bitand(A_uint, uint8(~B_seg * 255));
RGB_result = cat(3, R_ch, G_ch, B_ch);

figure; imshow(RGB_result); title('Найденные отрезки (выделены красным)');
```

3.5. Результаты работы Части 4

Все 5 отрезков сторон пятиугольника обнаружены и точно выделены по длине. Результат подсвечен красным цветом.



4. ВЫВОДЫ ПО РАБОТЕ

В ходе выполнения лабораторной работы:

1. Изучены механизмы работы градиентных операторов Собела и Канни. Подтверждено преимущество алгоритма Канни за счет немаксимального подавления и гистерезиса.

2. Освоено преобразование Хафа в канонической форме ($y = kx + b$) и определена причина пропадания вертикальных прямых из-за сингулярности параметров k и b.

3. Реализовано изотропное преобразование Хафа в нормальной форме ($\rho = x \cdot \cos \varphi + y \cdot \sin \varphi$), позволившее обнаружить все 5 граней пятиугольной фигуры.

4. Разработан алгоритм детектирования ограниченных отрезков с помощью фиксации крайних координат [Xmin, Ymin, Xmax, Ymax].

5. Применение таблиц LUT и ограничение сектора углов по фазе градиента γ позволило суммарно ускорить работу программы более чем в 80 раз.